Lab 3: Monte Carlo Methods

Morgan Cooper
MSDS 684 — Reinforcement Learning

March 30, 2026

Section 1: Project Overview

This lab builds upon earlier concepts like Markov Decision Processes (MDP) and Dynamic Programming (DP) by introducing Monte Carlo Simulations. Specifically, this lab uses Monte Carlo (MC) and Reinforcement Learning (RL) to learn to play blackjack. This is done by letting an agent play blackjack hands hundreds of thousands of times without knowning any rules, just observing the outcomes. MC RL follows a model-free approach, since the agent has zero initial knowledge of card probabilities, or dealer behavior. This contrasts well with DP since DP requires a complete model of the environment. This lab investigates the first-visit MC control with differeing epsilon policies.

MC methods fit value functions by estimating full rewards at the end of each episode, but only uses the first occurence of each state-action pair per episode, ensuring return samples are independent and unbiased (Sutton Barto, 2020). To balance exploration and exploitation RL with MC uses an epsilon greedy policy, which results in a a high chance of non-greedy actions at the beginning of training and decays to a very small chance of non-greedy actions late in training. This ensures the agent has explored all actions sufficiently to determine an optimal policy. This lab also uses on-policy MC control meaning the same epsilon-greedy policy that generates episodes is also the policy being improved.

The environment used in the lab follows:

Blackjack-v1 (Gymnasium):

- State space: discrete 3-tuple (player sum 4–21, dealer showing card 1–10, usable ace True/False) — approximately 400 unique states
- Action space: 2 discrete actions — hit (draw a card) or stick (stop drawing)
- Rewards: $+1$ for winning, -1 for losing, 0 for a draw — only awarded at the end of each hand
- Terminal conditions: player busts (exceeds 21), player sticks, or player receives a natural blackjack
- Uses Sutton & Barto rules: infinite deck, natural blackjack only counts without hitting

I expect the agent to sucessfully learn an optimal blackjack policy, but only if its given enough training time. Since I am familiar with Blackjack I am expecting the agent to learn quickly to stick when the value of the hand is high, and hit when it is low. In blackjack, if the player busts first the dealer doesnt have to reveal their cards, even if they too would have busted the player looses. Therfore, I dont expect win rate to get higher than 45

Section 2: Deliverables

GitHub Repository

`GitHub Repository: https://github.com/cooper-rm/monte-carlo-methods`

Implementation Summary

I implemented first-visit MC control with epsilon-greedy policies using Gymnasium's Blackjack-v1 environment. The primary training ran 500,000 episodes with $\epsilon = 0.1$ and $\gamma = 1.0$. The MC RL agent updated the values in a Q-table as it learned, utilizing incremental averaging. I also ran experiments with ranging epsilon values (0.01, 0.1, 0.3) and differing decay schedule algorithms (linear and exponential, both 0.3 to 0.01). Beyond this, I decided to run extended episodes to see if slower trainng made a substantial difference. These can be seen in Figures 1–4.

Key Results & Analysis

After training for 500,000 episodes, I collected and visualized the state value function $V(s) = \max_a Q(s, a)$ for each state across the usable and non-usable ace scenarios and compared it to the baseline decision for playing blackjack. There are two plot types, one for usable ace scenarios and one without. When we compare the chances of winning when the agent has a usable ace versus a non-usable ace, it appears that the chance of winning is higher overall when a usable ace exists. Both visuals present a cliff pattern where the chances of the player winning are substantially higher and increase rapidly as the agent's hand moves from 17 to 21. Below 17, the chance of winning is at or below zero in both scenarios.

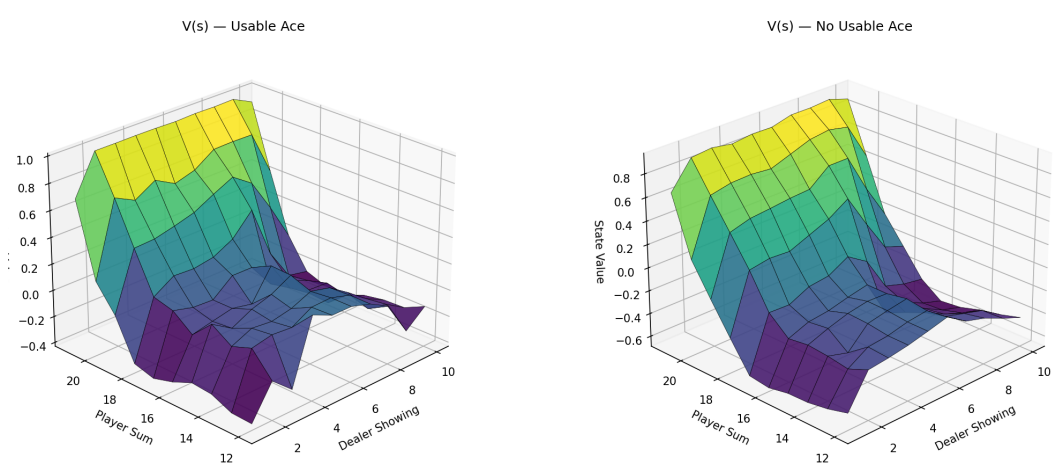

Figure 1: 3D surface plots of the learned state value function V(s) after 500,000 episodes of first-visit MC control. Left: states with a usable ace. Right: states without a usable ace. The high plateau at player sum 20–21 reflects near-certain wins, while the valley when the dealer shows 9–10 reflects the dealer's strength.

Figures 2 and 3 show the learned policy compared against the baseline strategy for both

usable ace and non-usable ace. Figure 2 shows the learned policy after 500k training steps and Figure 3 shows the learned policy after 1.5 million episodes. While the agent learns a good policy after only 500k steps, it is easy to see that the states that are likely not visited often are different between the baseline and the learned policy. However, after 1.5 million training steps the agent is nearly perfect. This exemplifies the tradeoff between longer training and accuracy, as there is a diminishing return to training as it gets longer.

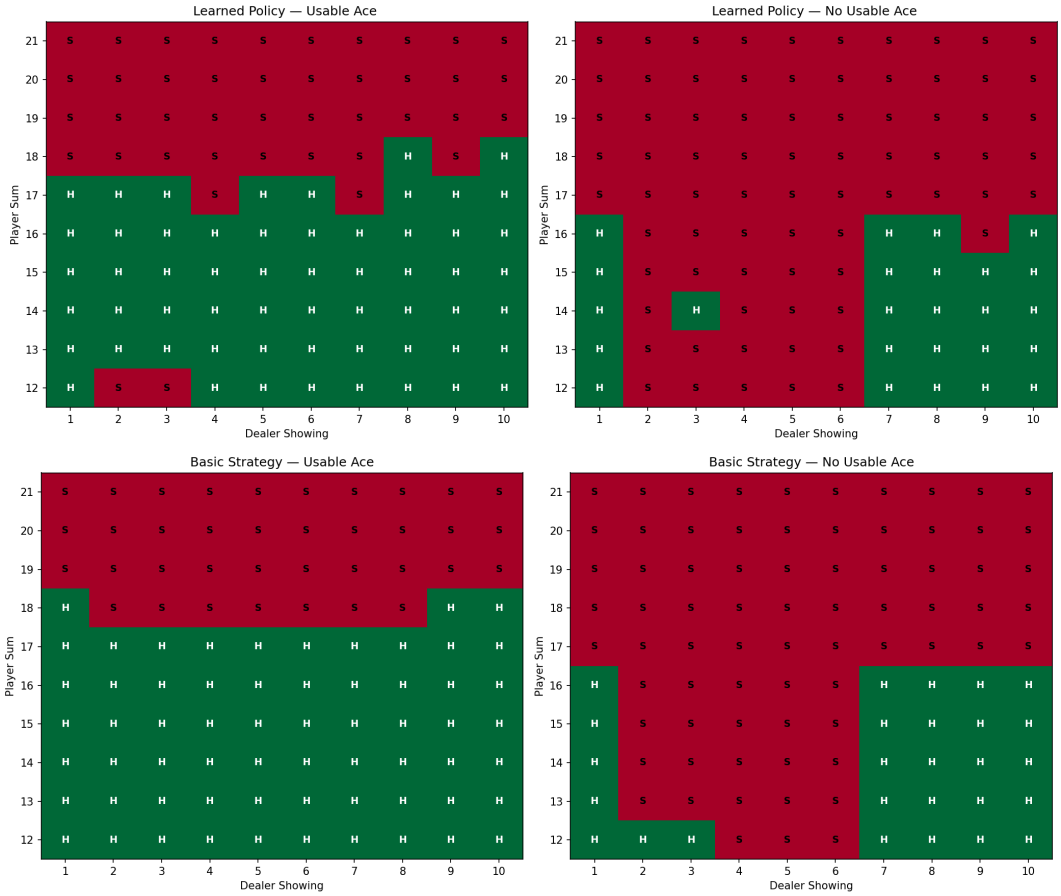

Figure 2: Top: Learned policy heatmaps after 500,000 episodes of first-visit MC control. Bottom: Basic Blackjack strategy reference. In both rows, left is usable ace and right is no usable ace. Green cells indicate stick (S), red cells indicate hit (H). The learned policy closely matches established basic strategy, with minor differences in borderline states such as soft 18 against strong dealer cards.

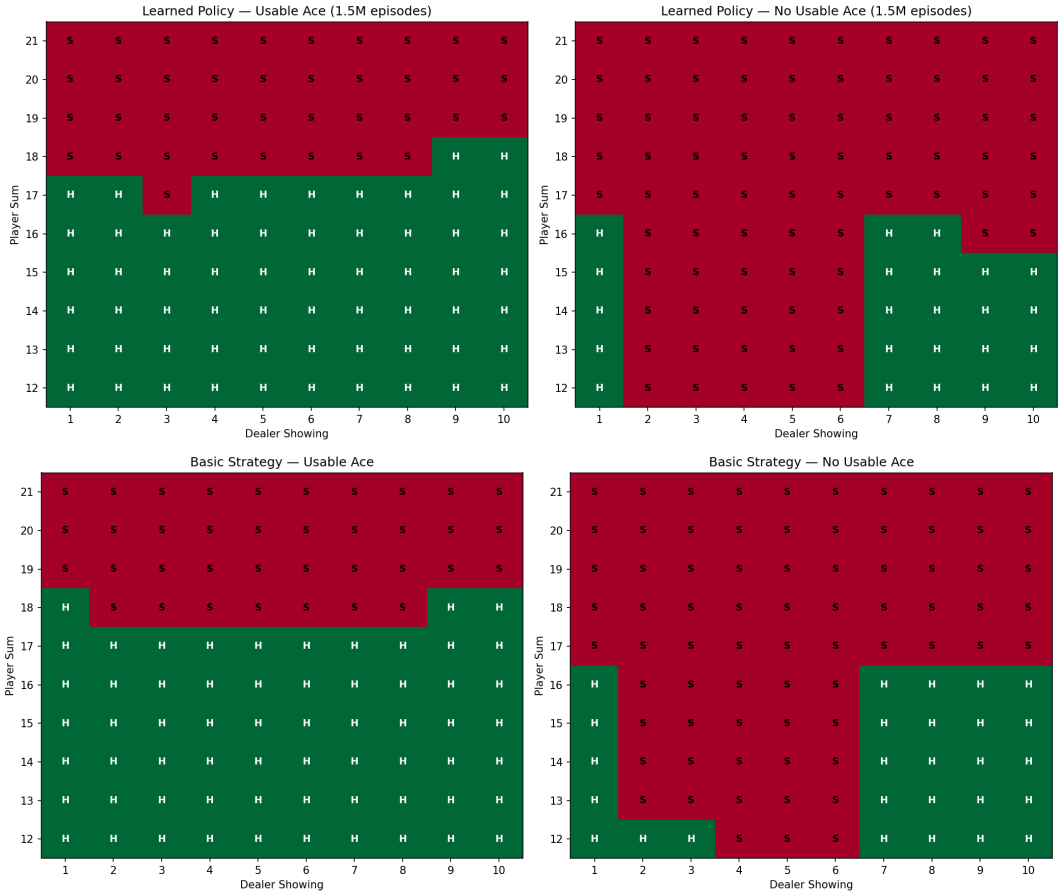

Figure 3: Top: Learned policy after 1,500,000 episodes with linear epsilon decay (0.3 to 0.01). Bottom: Basic Blackjack strategy reference. Green cells indicate stick (S), red cells indicate hit (H). The learned policy closely matches established basic strategy, with minor differences in borderline states such as soft 18 against strong dealer cards.

Multiple experiments were run comparing different epsilon values (0.01, 0.1, 0.3), decay schedules (linear, exponential), and episode lengths. Initially, all configurations were run for 500k episodes and then the best were rerun for 1.5 million episodes. From these experiments, we saw exponential decay have the highest win rate early on, but linear decay eventually surpassed it, just slightly. The constant $\epsilon = 0.3$ stays the lowest because too much random play prevents exploitation of optimal actions. The 1.5 million episode runs made mild improvements and were roughly the same as the best 500k run, but ended up fitting much closer to the baseline policy.

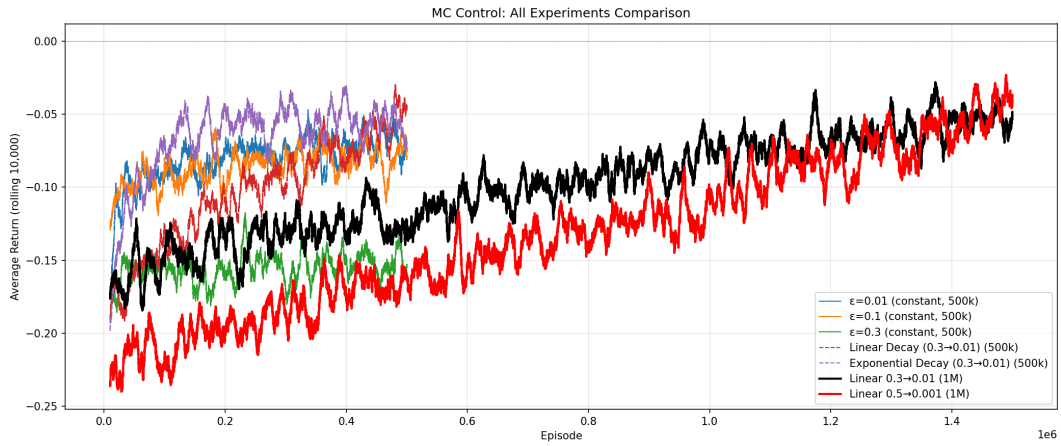

Figure 4: Learning curves comparing constant epsilon values and decay schedules. All curves show rolling 10,000-episode average returns. Decay schedules outperform constant epsilon by combining early exploration with late exploitation. Extended 1.5M-episode runs reach similar performance to the best 500k schedules, suggesting diminishing returns to additional training.

Section 3: AI Use Reflection

Initial Interaction

I find more and more each week that Claude is pretty good at getting things right the first time. I am unsure if this is because of improvements to Claude or improvements to the way I converse with it. Before I even started working in Claude, I opened a discussion with ChatGPT to help me learn the materials better. After that, I worked with Claude Code in a Jupyter notebook to develop a scaffolding for the project with markdown cells based on the directions. Once we began working on the code, I had Claude go through each line and explain the what and why of the codebase.

Iteration Cycle

Iteration 1: Understanding Backward Return Computation

After Claude implemented the Monte Carlo control training loop, I found myself a little confused on exactly how it worked, so I asked Claude to explain each line and discuss a clear example. Claude traced through a specific trajectory showing me how $G = reward + \gamma \cdot G$ propagates backward. This clarified a key insight: step 0's immediate reward was 0, but $G = 1.0$ because the backward pass propagates the future win to the decision that resulted in it. After grasping exactly how it works, I learned that we use backward-propagating reward to avoid the recursive and computationally expensive requirements of propagating reward using a forward pass.

Iteration 2: Wrong Cell Type for Part 8

When working through Part 8, which saves figures to be used in this report, Claude made

the mistake of writing code to a markdown cell instead of a Python cell. This was a simple error that I caught right away, so I reverted the changes and reprompted Claude to try again in the next cell after the markdown cell. This was a simple mistake, so it only required a single intervention.

Iteration 3: Experimenting with Epsilon Schedules

At first, I implemented experiments with 500k episode lengths. Since it appeared that the percentage of success for some configurations was still trending up, I wanted to see what would happen if I experimented further with 1 million episodes on the linear decay and a more aggressive $0.5 \rightarrow 0.001$ schedule. After seeing the results, I observed that the extra steps appeared to be a waste and the result was about the same. I then looked further at the baseline policy vs. the learned policy and realized the 500k steps were still not perfect. I ran the experiment again with 1.5 million steps, and visually the run appeared to be about the same outcome. However, when looking at the policy via the red and green grid and comparing it to the baseline, it was much closer than the 500k steps. This led to a clear understanding that there is a diminishing return on longer training, but longer training will get you closer to the end goal since it involves experimenting more with rare scenarios.

Critical Evaluation

Overall, Claude did a good job helping to complete this lab, since it only made one big error of writing to the wrong cell type. What was most helpful was first setting up an interview-style learning process via ChatGPT initially, then working through each step one at a time with Claude Code. At each step, I was able to work through the code line by line and understand the concepts at a high level. While I could have easily implemented the code without Claude's help, this process allowed me to focus on concepts more than syntax.

Learning Reflection

What was most helpful throughout this process was looking at the policy heatmaps against the known basic Blackjack strategy. Understanding exactly what the policy looked like and continuing to experiment with longer training demonstrated the Pareto distribution (80/20) cleanly. Most of the results came from early in the training, but fine-tuning took a lot more effort. Through this process, it was easy to see that Monte Carlo is viable in environments where there is no known model. It also made clear a new way of thinking when working with back-propagating values vs. forward-propagating values, since in unknown model environments it is a lot more computationally feasible.

Section 4: Speaker Notes

- **Problem:** Can we successfully train an RL agent to learn Blackjack with no rules?
- **Method:** We use first-visit Monte Carlo control with a linearly decaying epsilon-greedy policy.

- **Design choice:** We test multiple different epsilon values to optimize the agent's ability to learn.
- **Key result:** After 500k steps the agent learns to play, and after 1.5 million the agent's decisions are near optimal.
- **Insight:** How you schedule epsilon matters more than how many episodes you run.
- **Challenge:** The most difficult challenge in RL is monitoring exploration vs. exploitation and optimizing for the relationship.
- **Connection:** Monte Carlo, which functions without a known model, bridges the gap from Dynamic Programming, which requires a known model.

References

1. Sutton, R. S., & Barto, A. G. (2018). *Reinforcement learning: An introduction* (2nd ed.). MIT Press.
2. Anthropic. (2025). Claude Code [Large language model CLI tool]. `https://claude.ai`
3. OpenAI. (2025). ChatGPT [Large language model]. `https://chat.openai.com`